MKB403

Stop Scripting, Start Engineering: Building Production-Ready ML Apps

Python Programming for Machine Learning

Project-Based Learning Course

DOCKER

STREAMLIT

SCIKIT-LEARN

GIT WORKFLOWS

The Zillow Crash

2021

FINANCIAL LOSS

$500M+

WORKFORCE IMPACT

2,000+

Layoffs

STOCK COLLAPSE

-40%

Their Zestimate ML model failed when real-world market conditions shifted.

Is your code a product or a liability?

The Technical Autopsy

1

Over-fitting to Historical Data

Model trained on stable markets couldn't adapt to rapid price shifts during pandemic housing boom

2

Poor Data Quality & Outlier Handling

Insufficient exploratory analysis led to systematic pricing errors in edge cases

3

Lack of Human-in-the-Loop Evaluation

Automated predictions without rigorous model evaluation and domain expert validation

→ **This Course Focus: Deep EDA + Rigorous Model Evaluation**

Why Industry Hates Messy Notebooks

✗ THE MISTAKE

notebook_final_v3.ipynb
Only runs on your laptop

- Hardcoded paths and credentials
- No version control or collaboration
- Can't reproduce results

// Unemployable code

✓ INDUSTRY STANDARD

Version-controlled modules
Git workflows with proper commits

- Modular, testable functions
- Containerized
- Deployable anywhere

// Production-ready

The Full-Stack ML Engineer

Not Enough

```
import sklearn

model.fit(X, y)

model.predict(...)
```

Anyone can call a library

✓ Industry Wants

- Build ML pipelines in Python
- Deploy with containers
- Build Polished UIs
- Collaborate with Git workflows
- Validate with rigorous evaluation

Engineers who ship products, not just results

Logic vs. Data: Software 2.0

Connecting your prerequisites to production ML

PREREQUISITE 1

Algorithms

You write the logic

PREREQUISITE 2

Statistics

You analyze the data

↓

Machine Learning: Data Defines Logic

The algorithm learns patterns from data instead of you coding explicit rules

Orange

Visual prototyping

→

Python

Production engine

The Team as a Dev Unit

Team Approach

Gitmoji Commits

```
✨ feat: add new feature
🐛 fix: resolve bug
📝 docs: update README
```

Conventional Commits

Standardized commit messages that create readable project history and enable automated changelogs

Cookiecutter Data Science

Industry-standard project structure for reproducible data science workflows

Professional Git workflows from Day 1

The GitHub 'Resume'

Your repo is your proof of skill

For Informatics graduates, a GitHub repository following CCDS standards with a working Dockerfile speaks louder than any certificate.

EMPLOYERS SEE

- Clean commit history
- Professional project structure
- Documented code and README
- Deployable with one command

ml-project-repo/

```
├── data/
├── notebooks/
├── src/
|   ├── data.py
|   ├── features.py
|   ├── models.py
|   └── app.py
├── models/
├── Dockerfile
├── requirements.txt
├── README.md
└── .gitignore
```

// Cookiecutter Data Science

SPRINT 1

Discovery & Pitching

Weeks 1-8: Prove the model works before building the app

Weeks 1-2	**Environment Setup** Git workflows, CCDS structure, team formation
Weeks 3-4	**Visual ML with Orange** Rapid prototyping and workflow visualization
Weeks 5-6	**Deep EDA in Python** Data quality, outliers, feature engineering
Weeks 7-8	**Business Pitch (ETS)** **Present Proposal with model feasibility, EDA insights, and business value**

GOAL: Validate before you build

SPRINT 2

Engineering & Shipping

Weeks 9-16: Build and deploy the production app

Weeks 9-12	**Code 4 ML Algorithms** Classification, Regression, Clustering, Time Series
Weeks 13-14	**Streamlit UI Development** Interactive dashboard for model predictions
Week 15	**Pre-Production** Open Discussion for ML Project Delivery
Week 16	**App Fair (UAS)** **Demo live ML dashboard**

FINALE: Ship a production-ready ML product

The Grading Contract

COMPONENT 1

30%

Teamwork & Git

Professional workflows and collaboration

COMPONENT 2 (ETS)

30%

Mid-term Pitch

Business case and model validation

COMPONENT 3 (UAS)

40%

App Fair

Live demo of ML app

Key Metric: PROFESSIONALISM in code, commits, and collaboration

Rules of Engagement

AI as a Copilot: ChatGPT, Gemini, and Code Assistants

Allowed

- ✓ Use AI for debugging syntax errors
- ✓ Learn new libraries and concepts
- ✓ Generate boilerplate code structures
- ✓ Understand error messages

The Requirement

You must be able to explain your code logic and design decisions during the App Fair. Blind copy-paste = automatic fail.

Remember Zillow

Blind trust in algorithms without understanding the logic led to $500M+ in losses.

`Understanding > Copy-Paste`

AI is a tool, not a replacement for engineering thinking

TICKET #1

Initializing the System

Your first technical task starts now

1

Forming Team

3-4 members per team. Assign roles: Data Engineer, Model Engineer, UI Engineer, DevOps

2

Initialize GitHub Repo

Use Cookiecutter Data Science template to create professional project structure

3

Push First Commit

Use correct Gitmoji and Conventional Commit format:

```
🎉 init: initial project setup with CCDS structure
```

Due: End of Week 1

Your journey from script to product starts here